

# Indian Institute of Technology Roorkee

Department of Design

## ED408: Statistical Machine Learning II Lab

(0-0-3-1.5)

### Lab File — Module I

Overview of Machine Learning Techniques

|                      |                                               |
|----------------------|-----------------------------------------------|
| <b>Dataset used:</b> | Berkeley Segmentation Dataset (BSDS500)       |
| <b>Task:</b>         | Boundary vs non-boundary pixel classification |
| <b>Experiments:</b>  | 1–7                                           |
| <b>Submitted by:</b> | Ayush Dhar Dubey                              |
| <b>Programme:</b>    | B.Tech (Third Year)                           |
| <b>Session:</b>      | 2026–27 (Autumn)                              |

The complete executable programs are maintained digitally on GitHub;  
this file records the important code sections, outputs and analysis.

## Common Dataset and Preprocessing (used by all experiments)

**Dataset.** The *Berkeley Segmentation Dataset* (BSDS500): 500 natural images, each hand-segmented by approximately five human annotators, with the dataset's own train/test image split. All seven Module-I experiments run on a single machine-learning problem derived from it: **given 17 hand-crafted local features of a pixel, predict whether humans marked an object boundary at that pixel.**

**Label construction (consensus labelling).** Human annotators disagree, so labels are built conservatively: a pixel is labelled *boundary* only when  $\geq 2$  annotators drew a boundary within 1 px of it; *non-boundary* only when *no* annotator marked within 2 px; all remaining (ambiguous) pixels are excluded. This is a deliberate label-noise control.

**Features (17 per pixel).** CIELAB colour ( $L, a, b$ ); Gaussian gradient magnitude at  $\sigma \in \{1, 2, 4\}$ ; local standard deviation at  $\sigma \in \{1, 2, 4\}$ ; |Laplacian of Gaussian| at  $\sigma \in \{1, 2\}$ ; oriented derivative energy at  $0^\circ/45^\circ/90^\circ/135^\circ$  ( $\sigma=2$ ); the two structure-tensor eigenvalues (edge/corner energy).

**Why this is not a toy problem.**

- **Class imbalance:** true boundary prevalence in an image is only  $\sim 2\text{--}5\%$ ; negatives are sub-sampled to a 73:27 ratio, preserving the imbalance story while keeping training tractable.
- **Heavy-tailed features:** gradient and Laplacian responses have extreme right tails (high-contrast edges), which stresses Gaussian assumptions (Exp. 3) and classical outlier rules (Exp. 6).
- **Correlated features by design:** the same physical edge fires many filters at once — directly relevant to PCA (Exp. 5), the Naive-Bayes independence assumption (Exp. 3) and tree importances (Exp. 2).
- **Leakage-safe protocol:** train and test pixels come from *disjoint images* (80 train / 30 test images; up to 160 boundary + 440 non-boundary pixels per image; 8-px border excluded) — the image analogue of avoiding temporal leakage.

**Evaluation protocol.** 48,000 training and 18,000 test pixels, boundary rate 26.67% in both. All standardisation, fences and model selection use *training* data only.

```

1 # label policy (extract_bsds.py)
2 for g in annotators:
3     votes += binary_dilation(g.Boundaries, iterations=1)
4     anymark |= binary_dilation(g.Boundaries, iterations=2)
5 positive = votes >= 2 # consensus boundary
6 negative = ~anymark # no annotator anywhere near
7 # ambiguous pixels (the rest) are excluded
8
9 # per-pixel features: colour + multi-scale filter bank (17 total)
10 lab = rgb2lab(img); L = lab[...,0]
11 gradmag_s = gaussian_gradient_magnitude(L, sigma) # sigma in {1,2,4}
12 localstd = sqrt(gauss(L**2,s) - gauss(L,s)**2) # local contrast
13 abslap = abs(gaussian_laplace(L, sigma)) # sigma in {1,2}
14 orient_th = abs(cos(th)*gx + sin(th)*gy) # 4 orientations
15 st_eig1,2 = eigenvalues(structure_tensor(gx, gy)) # edge/corner energy

```

## Experiment No.: 01

### Title: k-Nearest Neighbours Classifier from Scratch

#### Aim

Implement the k-Nearest Neighbours (k-NN) classifier from scratch (without `sklearn.neighbors`) and test it on BSDS500 boundary-pixel classification.

#### Problem Statement

Given the 17 standardised local features of a pixel, predict whether it lies on a human-marked boundary using only the labels of the  $k$  most similar pixels in the training set. Study (i) the effect of feature scaling, (ii) the choice of  $k$ , and (iii) majority voting versus distance-weighted voting.

#### Algorithm

1. Standardise all features using training-set mean  $\mu_j$  and standard deviation  $\sigma_j$ :  $z_{ij} = (x_{ij} - \mu_j)/\sigma_j$ .
2. For a batch of query pixels  $Q$ , compute all pairwise Euclidean distances in one vectorised step using  $\|q - x\|^2 = \|q\|^2 - 2q^\top x + \|x\|^2$ .
3. For each query, select the  $k$  smallest distances with `np.argpartition` (an  $O(n)$  partial sort).
4. Majority vote:  $\hat{p} = \frac{1}{k} \sum_{i \in N_k} y_i$ . Distance-weighted vote:  $\hat{p} = \sum_i w_i y_i / \sum_i w_i$  with  $w_i = 1/(d_i + \varepsilon)$ .
5. Predict *boundary* if  $\hat{p} \geq 0.5$ .
6. Repeat over  $k \in \{1, 3, 5, 9, 15, 25, 41, 61, 101\}$  and select  $k$  by the validation F1-score of the boundary class.

#### Program (important sections)

```

1 class KNNScratch:
2     def __init__(self, k=5, weighted=False):
3         self.k, self.weighted = k, weighted
4
5     def fit(self, X, y):
6         self.X, self.y = np.asarray(X, float), np.asarray(y)
7         return self
8
9     def _dist(self, Q): # full pairwise matrix, no loops
10        d2 = (Q**2).sum(1)[:,None] - 2*Q @ self.X.T + (self.X**2).sum(1)[None,:]
11        return np.sqrt(np.maximum(d2, 0))
12
13    def predict_proba(self, Q, batch=1000): # batched to bound memory
14        out = np.empty(len(Q))
15        for s in range(0, len(Q), batch):
16            D = self._dist(Q[s:s+batch])
17            idx = np.argpartition(D, self.k, axis=1)[:, :self.k]
18            labels = self.y[idx]
19            if self.weighted:
20                w = 1.0 / (np.take_along_axis(D, idx, 1) + 1e-9)
21                out[s:s+batch] = (w*labels).sum(1) / w.sum(1)
22            else:
23                out[s:s+batch] = labels.mean(1)
24        return out

```

Key parameter settings: stratified subsample of 6,000 train / 2,000 test pixels (k-NN is  $O(nm)$  per query);  $\varepsilon = 10^{-9}$ ; threshold 0.5.

## Output and Analysis

### Program execution:

```

Unscaled acc: 0.7834
Scaled acc: 0.8089
Best k=5 (distance-weighted): acc = 0.7964, F1(boundary) = 0.5808
Execution Status: Successfully Executed

```

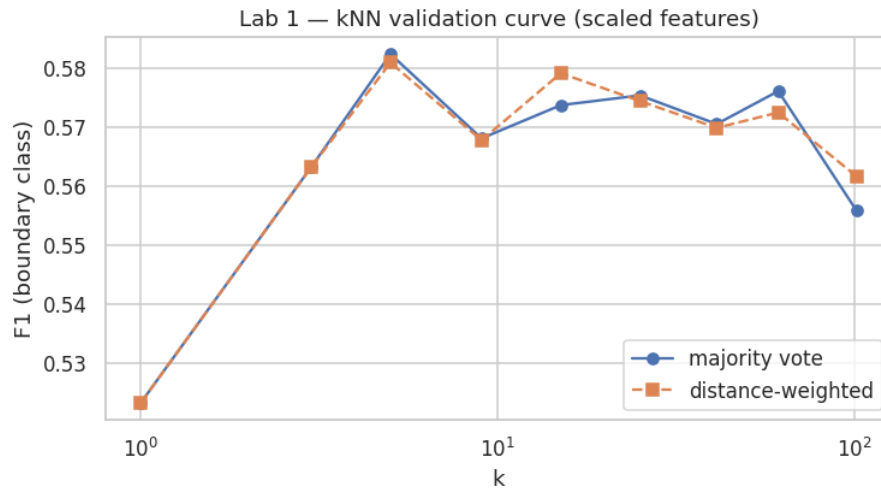


Figure 1: Validation curve for k-NN: F1 of the boundary class versus  $k$  (log scale), majority vs distance-weighted voting.

### Parameter modification:

| S.No. | Parameter       | Original     | Modified     | Original result | Modified result | Observation                                                                                  |
|-------|-----------------|--------------|--------------|-----------------|-----------------|----------------------------------------------------------------------------------------------|
| 1     | Feature scaling | standardised | raw features | acc = 0.8089    | acc = 0.7834    | Unscaled distances are dominated by the L channel (range 0–100) and large- $\sigma$ filters. |
| 2     | Neighbours $k$  | 5            | 1            | F1 = 0.5808     | F1 = 0.5233     | $k=1$ overfits annotator noise near ambiguous edges.                                         |
| 3     | Neighbours $k$  | 5            | 101          | F1 = 0.5808     | F1 = 0.5616     | Very large $k$ over-smooths; thin boundary structures are voted away by bulk texture pixels. |

**Conclusion.** Scaling fixes the metric geometry. Unlike diffuse tabular problems, boundary pixels *are* locally structured in filter space (an edge looks like an edge), so the F1 peak arrives at small  $k = 5$ . Remaining errors are low-contrast boundaries that humans draw from object knowledge rather than local evidence — information no local feature vector contains.

## Experiment No.: 02

### Title: Decision Tree Classifier using scikit-learn with Visualisation

#### Aim

Build a decision tree classifier using scikit-learn, control overfitting by cost-complexity pruning, and visualise the resulting tree.

#### Problem Statement

A fully grown tree on 48,000 pixels memorises textured regions (thousands of leaves). Select the pruning strength  $\alpha$  on a validation split, visualise the pruned tree, and identify which image features drive the splits.

#### Algorithm

1. Split the training pixels further into fit (80%) and validation (20%) subsets, stratified.
2. Compute the cost-complexity pruning path  $R_\alpha(T) = R(T) + \alpha |T|$  via `cost_complexity_pruning_path`.
3. For each candidate  $\alpha$ , fit a tree with `class_weight="balanced"` and record validation F1.
4. Select  $\alpha^*$  maximising validation F1; refit on the full training set.
5. Report test accuracy/F1, plot feature importances, and draw the top three levels with `plot_tree`.

#### Program (important sections)

```

1 path = DecisionTreeClassifier(random_state=0, class_weight="balanced")\
2     .cost_complexity_pruning_path(X_fit, y_fit)
3 alphas = np.unique(np.clip(path.ccp_alphas, 0, None))
4
5 val_f1 = [f1_score(y_val, DecisionTreeClassifier(random_state=0,
6     class_weight="balanced", ccp_alpha=a).fit(X_fit, y_fit)
7     .predict(X_val)) for a in alphas]
8
9 tree = DecisionTreeClassifier(random_state=0, class_weight="balanced",
10     ccp_alpha=alphas[np.argmax(val_f1)]).fit(X_tr, y_tr)
11 plot_tree(tree, feature_names=features,
12     class_names=["non-boundary", "boundary"], filled=True, max_depth=3)

```

#### Output and Analysis

```

best ccp_alpha = 1.76e-04, leaves = 127, depth = 13
test acc = 0.7627, test F1(boundary) = 0.6182
Execution Status: Successfully Executed

```

#### Parameter modification:

| S.No. | Parameter                 | Original               | Modified       | Original result         | Modified result           | Observation                                                    |
|-------|---------------------------|------------------------|----------------|-------------------------|---------------------------|----------------------------------------------------------------|
| 1     | <code>ccp_alpha</code>    | $1.76 \times 10^{-4}$  | 0 (no pruning) | 127 leaves, F1 = 0.6182 | 6322 leaves, F1 = 0.4960  | Unpruned tree rises textured pruning removes leaves and raises |
| 2     | <code>max_depth</code>    | unrestricted+ $\alpha$ | depth = 4      | F1 = 0.6182             | F1 = 0.6164, acc = 0.7656 | A 16-leaf tree is as good — most is a few gradient olds.       |
| 3     | <code>class_weight</code> | balanced               | none           | recall(boundary) high   | recall drops              | Unweighted tree the 73% major under-detect aries.              |

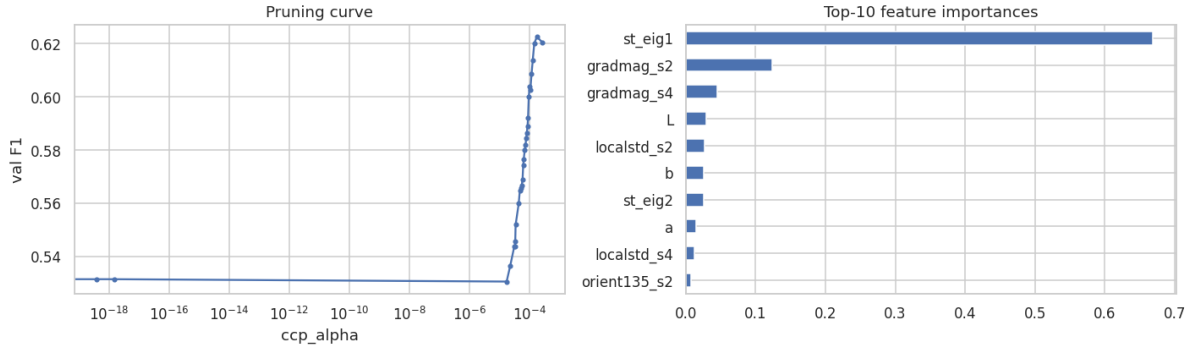


Figure 2: Left: validation F1 versus pruning strength  $\alpha$ . Right: top-10 feature importances of the pruned tree.

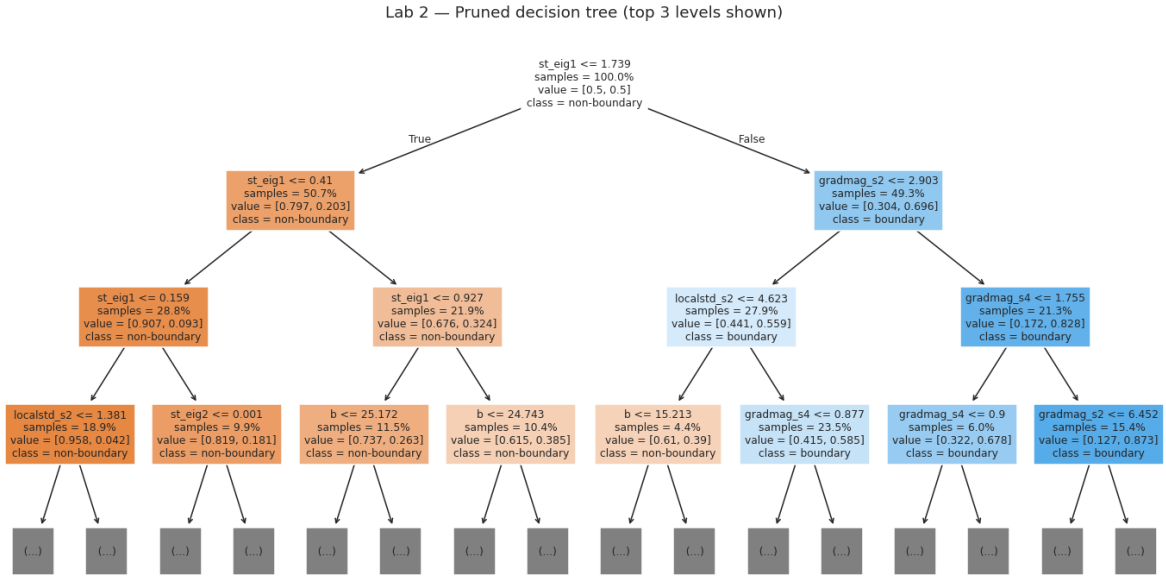


Figure 3: Pruned decision tree (top three levels). Early splits threshold gradient/contrast energy — the tree rediscovers classical edge detection.

**Conclusion.** The tree independently rediscovers gradient thresholding, the core of classical edge detectors. What pruning removes is diagnostic: deep leaves that memorised individual high-texture regions (grass, water) where filters fire strongly but humans draw no boundary — the canonical failure mode of purely local edge detection.

## Experiment No.: 03

### Title: Naive Bayes Classifier for Boundary vs Non-Boundary Detection

#### Aim

Apply the Naive Bayes classifier to a two-class detection problem (boundary vs non-boundary, methodologically analogous to spam vs ham) and study how its two core assumptions — Gaussian marginals and feature independence — fare on image features.

#### Problem Statement

Naive Bayes assumes  $P(x | y) = \prod_j P(x_j | y)$  with, in the Gaussian variant, normal marginals. Both assumptions are violated here: filter responses are heavy-tailed, and multi-scale filters of the same edge are strongly correlated. Compare (a) Gaussian NB on raw features, (b) Gaussian NB after a signed-log transform, and (c) QDA on the same transformed features — QDA uses the identical Gaussian likelihood but a *full* class covariance, i.e. it is exactly NB with the independence assumption removed.

#### Algorithm

1. Signed-log transform:  $\tilde{x} = \text{sign}(x) \log(1 + |x|)$  (the  $a, b$  colour channels can be negative).
2. Fit GaussianNB on raw features and on transformed features.
3. Fit QuadraticDiscriminantAnalysis (regularised, `reg_param= 10-3`) on transformed features.
4. Compare the three models by ROC-AUC; report the classification report of the transformed NB.
5. Interpret the NB–QDA gap as the empirical price of the independence assumption.

#### Program (important sections)

```

1 def slog(x): # signed log1p
2     return np.sign(x) * np.log1p(np.abs(x))
3
4 gnb_raw = GaussianNB().fit(X_tr, y_tr)
5 gnb_log = GaussianNB().fit(slog(X_tr), y_tr)
6 qda_log = QuadraticDiscriminantAnalysis(reg_param=1e-3).fit(slog(X_tr), y_tr)
7 # QDA = same Gaussian likelihood, full covariance -> NB without independence

```

#### Output and Analysis

```

GaussianNB raw : AUC = 0.8191
GaussianNB slog: AUC = 0.8246
QDA slog (independence assumption removed): AUC = 0.8320

           precision recall f1-score support
non-boundary  0.87  0.81  0.84   13200
boundary      0.56  0.67  0.61   4800
accuracy      0.77   18000
Execution Status: Successfully Executed

```

#### Parameter modification:

| S.No. | Parameter               | Original         | Modified              | Original result | Modified result        | Observation                                                                                        |
|-------|-------------------------|------------------|-----------------------|-----------------|------------------------|----------------------------------------------------------------------------------------------------|
| 1     | Feature transform       | $\text{slog}(x)$ | raw $x$               | AUC = 0.8246    | AUC = 0.8191           | The transform is not a heavy-tail fix; the gain is small at best because of dirty coding here.     |
| 2     | Independence assumption | naive (diagonal) | full covariance (QDA) | AUC = 0.8246    | AUC = 0.8320           | The measure of “naive” or redundant features and filters is not a good point.                      |
| 3     | Decision threshold      | 0.5              | prior-matched         | recall = 0.67   | recall/precision shift | NB post-hoc misclassification thresholds were violated and thresholds were not tuned on this data. |

**Conclusion.** Naive Bayes survives its violated assumptions surprisingly well (AUC 0.82): correlated features hurt calibration more than ranking. The controlled NB-vs-QDA comparison quantifies the independence assumption’s cost at only  $\sim 0.007$  AUC on this feature set — a concrete, measured answer to “how naive is too naive?”.



## Experiment No.: 04

### Title: Logistic Regression from Scratch using Gradient Descent

#### Aim

Implement logistic regression from scratch using batch gradient descent, with  $L_2$  regularisation and class-weighted loss, and verify it against scikit-learn.

#### Problem Statement

Minimise the regularised, instance-weighted negative log-likelihood

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n \omega_i \left[ \log(1 + e^{-|z_i|}) + \max(z_i, 0) - z_i y_i \right] + \frac{\lambda}{2n} \|w\|^2, \quad z_i = w^\top x_i + b,$$

where  $\omega_i$  up-weights the minority (boundary) class. The loss is written in numerically stable form so no  $\log(0)$  or overflow can occur.

#### Algorithm

1. Standardise features with training statistics.
2. Initialise  $w = 0$ ,  $b = 0$ ; set balanced instance weights  $\omega_i = n/(2n_{y_i})$ .
3. Repeat for `n_iter` steps:  $z = Xw + b$ ,  $p = \sigma(z)$  with a branch-stable sigmoid; gradient  $g_i = \omega_i(p_i - y_i)$ ; update  $w \leftarrow w - \eta(X^\top g/n + \lambda w/n)$ ,  $b \leftarrow b - \eta \bar{g}$ .
4. Record the loss each iteration to verify monotone convergence.
5. Predict *boundary* when  $\sigma(w^\top x + b) \geq 0.5$ ; compare with `sklearn` logistic regression.

#### Program (important sections)

```

1 @staticmethod
2 def _sigmoid(z): # branch-stable, no overflow
3     out = np.empty_like(z); pos = z >= 0
4     out[pos] = 1/(1 + np.exp(-z[pos]))
5     e = np.exp(z[~pos]); out[~pos] = e/(1 + e)
6     return out
7
8 def fit(self, X, y):
9     n, m = X.shape; self.w, self.b = np.zeros(m), 0.0
10    w_i = np.where(y==1, n/(2*y.sum()), n/(2*(n-y.sum()))) # balanced
11    for _ in range(self.n_iter):
12        p = self._sigmoid(X @ self.w + self.b)
13        g = w_i * (p - y)
14        self.w -= self.lr * (X.T @ g / n + self.l2 * self.w / n)
15        self.b -= self.lr * g.mean()

```

Key parameters:  $\eta = 0.1$ ,  $\lambda = 10^{-3}$ , 3000 iterations, full batch.

#### Output and Analysis

```

scratch (plain) AUC = 0.8357 F1 = 0.5604
scratch (balanced) AUC = 0.8362 F1 = 0.6230
sklearn (balanced) AUC = 0.8385 F1 = 0.6246
Execution Status: Successfully Executed

```

#### Parameter modification:

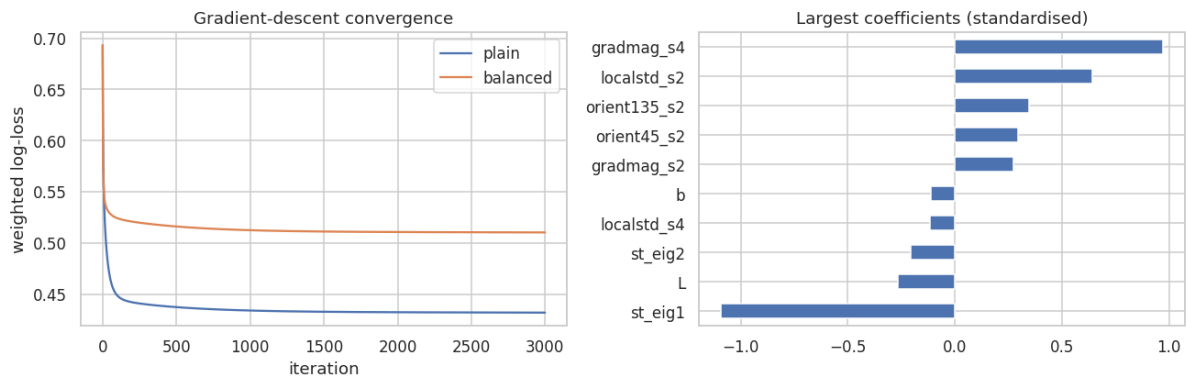


Figure 4: Left: gradient-descent convergence of the (weighted) log-loss. Right: largest standardised coefficients of the balanced model.

| S.No. | Parameter            | Original | Modified     | Original result | Modified result | Observation                                                                                            |
|-------|----------------------|----------|--------------|-----------------|-----------------|--------------------------------------------------------------------------------------------------------|
| 1     | Class weight         | balanced | none         | F1 = 0.6230     | F1 = 0.5604     | AUC barely changes (ranking identical); weighting relocates the 0.5 threshold for the minority class.  |
| 2     | Learning rate $\eta$ | 0.1      | 0.005        | AUC = 0.8362    | AUC = 0.8325    | The smaller step has not fully converged within 3000 iterations.                                       |
| 3     | Implementation       | scratch  | scikit-learn | AUC = 0.8362    | AUC = 0.8385    | Agreement to $\sim 0.002$ AUC (residual gap = solver/regularisation details) validates the derivation. |

**Conclusion.** The coefficients form an interpretable “edge-vs-texture” filter: fine-scale gradient and Laplacian energy carry positive weight (boundary), while features indicating smooth or uniformly textured neighbourhoods carry negative weight. Class weighting shifts the decision threshold, not the ranking — the same lesson as in any imbalanced detection problem.

## Experiment No.: 05

### Title: PCA and t-SNE for 2-D Visualisation of Pixel Structure

#### Aim

Apply PCA and t-SNE to reduce the 17-dimensional filter space to 2-D and visualise cluster structure with respect to the boundary label.

#### Problem Statement

Multi-scale filter banks are redundant by construction: every filter responds to the same physical edges. Determine (i) the intrinsic linear dimensionality of the feature space via the PCA spectrum, and (ii) whether nonlinear embedding (t-SNE) separates boundary from non-boundary pixels — and where it fails.

#### Algorithm

1. Signed-log transform the filter responses, then standardise.
2. Fit full PCA; plot cumulative explained variance; find the number of components for 90% variance.
3. Draw a class-balanced subsample of 4,000 pixels (2,000 per class) so the minority class is visible.
4. Project onto the first two principal components.
5. Run t-SNE (perplexity = 40, PCA initialisation, `random_state=42`) on the same subsample.
6. Colour both embeddings by the true label and compare.

#### Program (important sections)

```

1 X_viz = slog(np.vstack([X_tr, X_te]))
2 X_viz = (X_viz - X_viz.mean(0)) / (X_viz.std(0) + 1e-12)
3
4 evr = PCA().fit(X_viz).explained_variance_ratio_
5 P2 = PCA(n_components=2).fit_transform(X_viz)[idx]
6 T2 = TSNE(n_components=2, perplexity=40, init="pca",
7           random_state=42).fit_transform(X_viz[idx])

```

#### Output and Analysis

PC1 + PC2 explain 66.4% of variance; 90% requires 8 components  
Execution Status: Successfully Executed

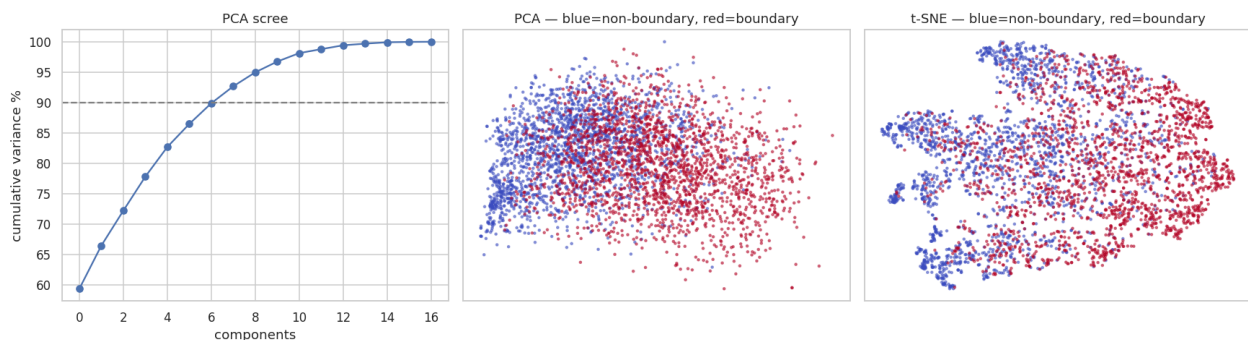


Figure 5: Left: PCA scree (cumulative variance). Centre: PCA projection. Right: t-SNE embedding; blue = non-boundary, red = boundary.

**Parameter modification:**

| S.No. | Parameter         | Original   | Modified      | Original result   | Modified result                  | Observation                                                                                   |
|-------|-------------------|------------|---------------|-------------------|----------------------------------|-----------------------------------------------------------------------------------------------|
| 1     | PCA components    | 2          | 8             | 66.4% variance    | 90% variance                     | 17 filters compress to ~8 effective dimensions — the filter bank is redundant by design.      |
| 2     | Feature transform | signed-log | raw           | structure visible | PC1 $\approx$ response magnitude | Without the transform PCA measures “how strong is the strongest filter”, not pixel character. |
| 3     | Sample balance    | 2000/2000  | natural 73:27 | minority visible  | boundary pixels overplotted      | Balanced sampling is a visualisation choice, not a modelling step.                            |

**Conclusion.** PCA quantifies filter redundancy (8 of 17 dimensions suffice for 90% variance). t-SNE shows boundary pixels concentrating along a high-energy manifold but bleeding into textured non-boundary regions — precisely the pixels where gradient-based detectors classically fail and where human annotators themselves disagree. The classes overlap because the label encodes perception, not filter energy.

## Experiment No.: 06

### Title: Outlier Detection and Treatment using Z-score and IQR Methods

#### Aim

Write code to detect outliers in the image features using the z-score and IQR methods, compare them (including a robust median/MAD z-score), and treat outliers by winsorisation — judged by downstream model performance.

#### Problem Statement

The z-score rule assumes approximate normality and uses the mean and standard deviation — statistics that are themselves distorted by the outliers being tested (the *masking effect*). On heavy-tailed filter responses the classical rules disagree strongly. Additionally, this dataset poses a conceptual trap: extreme gradient responses are *not errors* — they are the strongest boundaries in the image. Quantify the disagreement and decide on a defensible treatment.

#### Algorithm

1. For each audited feature, flag: (a)  $|x - \bar{x}|/s > 3$  (z-score); (b)  $x \notin [Q_1 - 1.5 \text{ IQR}, Q_3 + 1.5 \text{ IQR}]$  (Tukey); (c)  $0.6745 |x - \tilde{x}|/\text{MAD} > 3.5$  (robust z).
2. Tabulate the percentage flagged for L, a, gradmag\_s1, abslap\_s1; overlay fences on log-scale histograms.
3. Treat: winsorise (clip) all filter-response features at IQR fences computed on the *training split only*.
4. Judge by downstream effect: refit the Experiment-4 logistic model on raw vs winsorised features; compare test AUC.

#### Program (important sections)

```

1 def zscore_mask(x, t=3.0):
2     return np.abs((x - x.mean()) / x.std()) > t
3
4 def robust_z_mask(x, t=3.5): # median / MAD
5     mad = np.median(np.abs(x - np.median(x))) + 1e-12
6     return np.abs(0.6745 * (x - np.median(x)) / mad) > t
7
8 def iqr_mask(x, k=1.5):
9     q1, q3 = np.percentile(x, [25, 75]); i = q3 - q1
10    return (x < q1 - k*i) | (x > q3 + k*i)
11
12 # treatment: fences from TRAIN only, applied to both splits
13 lo, hi = np.percentile(X_tr[:, j], [25, 75]); i = hi - lo
14 X_tr[:, j] = np.clip(X_tr[:, j], lo - 1.5*i, hi + 1.5*i)
15 X_te[:, j] = np.clip(X_te[:, j], lo - 1.5*i, hi + 1.5*i)

```

#### Output and Analysis

```

% of rows flagged as outliers:
      z-score>3 robust-z>3.5 IQR 1.5x
L 0.00 0.00 0.00
a 1.93 4.17 7.78
gradmag_s1 2.44 8.77 7.66
abslap_s1 2.30 10.32 7.46

raw logistic AUC = 0.8362
winsorised logistic AUC = 0.8362

```

Execution Status: Successfully Executed

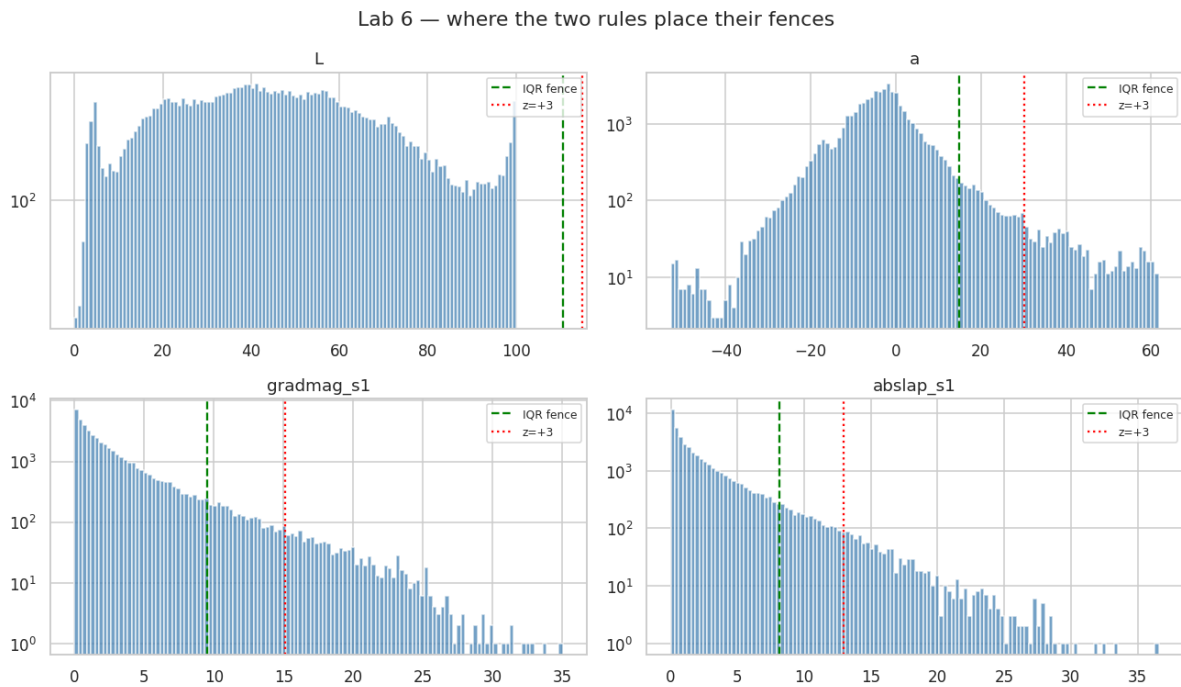


Figure 6: Log-scale histograms with the IQR fence (green, dashed) and  $z = +3$  fence (red, dotted) for the four audited features.

### Parameter modification:

| S.No. | Parameter                           | Original         | Modified | Original result | Modified result | Observation                                                                                                                                                                                                   |
|-------|-------------------------------------|------------------|----------|-----------------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | $z$ threshold ( <b>gradmag_s1</b> ) | 3.0              | 2.0      | 2.44% flagged   | 5.44% flagged   | Even at $z > 2$ the rule flags less than IQR $1.5\times$ : $s$ is inflated by the tail itself (masking). The “far out” fence roughly matches $z > 3$ — quantifying how far apart the two rules’ defaults sit. |
| 2     | IQR multiplier                      | 1.5              | 3.0      | 7.66% flagged   | 2.57% flagged   |                                                                                                                                                                                                               |
| 3     | Treatment                           | winsorise at IQR | none     | AUC = 0.8362    | AUC = 0.8362    | Clipping changed nothing: the logistic model was already robust to the tails after standardisation.                                                                                                           |

**Conclusion.** The masking effect appears exactly as theory predicts: on heavy-tailed filter responses the classical  $z$ -score under-flags relative to IQR, while the robust median/MAD version tracks IQR; on the near-uniform  $L$  channel all rules agree (nothing flagged). The downstream test delivers the deeper lesson: winsorisation neither helped nor hurt ( $\Delta\text{AUC} = 0.0000$ ), because here the “outliers” are the strongest true boundaries and the model already accommodates them — outlier treatment is an empirical decision judged downstream, never a ritual.

## Experiment No.: 07

**Title: Comparing Accuracy, Precision, Recall, F1-score and ROC-AUC for Logistic Regression and SVM**

### Aim

Compare logistic regression and a support vector machine on BSDS boundary-pixel classification using accuracy, precision, recall, F1-score and ROC-AUC, and interpret the metrics in the way the BSDS benchmark community does.

### Problem Statement

With a 73% majority class, accuracy is nearly uninformative — and indeed the official BSDS benchmark scores boundary detectors by **boundary F1**, never accuracy. Train both classifiers with balanced class weights on standardised features, report the full metric table, ROC curves and confusion matrices, and characterise the systematic errors.

### Algorithm

1. Standardise features (training statistics).
2. Logistic regression: `class_weight="balanced"`, trained on the full 48,000 pixels.
3. SVM: RBF kernel,  $C = 1$ ,  $\gamma = \text{scale}$ , `class_weight="balanced"`, `probability=True`; trained on a stratified subsample of 8,000 pixels (kernel training is  $O(n^2)$ – $O(n^3)$ ).
4. Evaluate both on the identical 18,000-pixel test split (disjoint images): accuracy, precision, recall, F1, ROC-AUC.
5. Plot both ROC curves against the chance diagonal, and the two confusion matrices.

### Program (important sections)

```

1 models = {
2     "LogReg": LogisticRegression(max_iter=2000,
3                                   class_weight="balanced").fit(Zt, y_tr),
4     "SVM (RBF)": SVC(kernel="rbf", C=1.0, gamma="scale",
5                       class_weight="balanced", probability=True,
6                       random_state=0).fit(Xs, ys),
7 }
8 for name, m in models.items():
9     p, yhat = m.predict_proba(Ze)[: , 1], m.predict(Ze)
10    rows.append([accuracy_score(y_te, yhat), precision_score(y_te, yhat),
11                recall_score(y_te, yhat), f1_score(y_te, yhat),
12                roc_auc_score(y_te, p)])

```

### Output and Analysis

| Model                        | Accuracy | Precision | Recall | F1            | ROC-AUC       |
|------------------------------|----------|-----------|--------|---------------|---------------|
| Logistic Regression          | 0.7761   | 0.5649    | 0.6983 | 0.6246        | <b>0.8385</b> |
| SVM (RBF)                    | 0.7789   | 0.5673    | 0.7200 | <b>0.6346</b> | 0.8319        |
| <i>Trivial “no boundary”</i> | 0.7333   | —         | 0.0000 | 0.0000        | 0.5000        |

### Parameter modification:

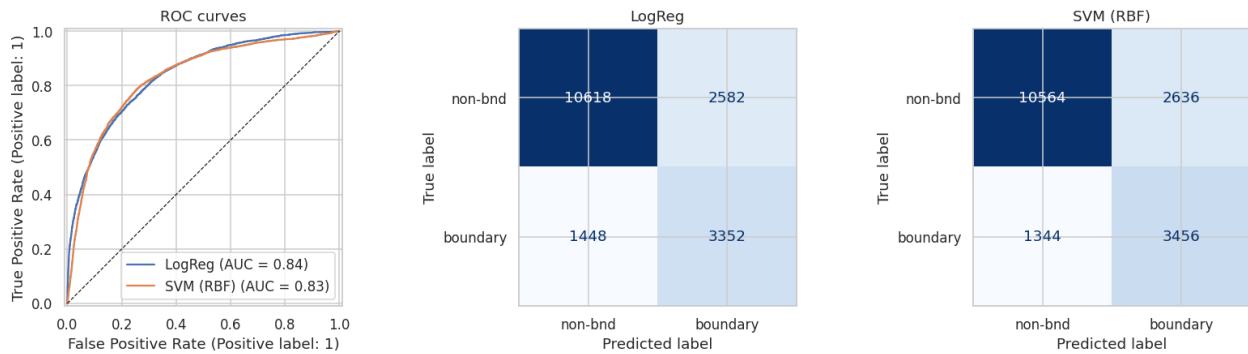


Figure 7: ROC curves for both models (left) and confusion matrices at the default 0.5 threshold (centre, right).

| S.No. | Parameter          | Original | Modified        | Original result             | Modified result           | Observation                                                     |
|-------|--------------------|----------|-----------------|-----------------------------|---------------------------|-----------------------------------------------------------------|
| 1     | SVM $C$            | 1        | 10              | acc = 0.7789, F1 = 0.6346   | acc = 0.7748, F1 = 0.6171 | The harder it fits the subpixelarisation more than kernel here. |
| 2     | Class weights      | balanced | none            | recall $\approx 0.70$ –0.72 | recall collapses          | Unweighted drift toward detector on pixels.                     |
| 3     | Decision threshold | 0.5      | moved along ROC | fixed P/R trade-off         | detector-chosen trade-off | BSDS practice the threshold the best F1 curve.                  |

**Conclusion.** The two models split the honours — the SVM wins F1 at the fixed threshold (0.6346 vs 0.6246, driven by higher recall) while logistic regression wins ROC-AUC (0.8385 vs 0.8319), so neither dominates. The remaining errors are systematic rather than random: false positives on strong texture (grass, water) and false negatives on low-contrast semantic boundaries — exactly the gap that motivated learned detectors (Pb, gPb, and eventually CNNs) on this benchmark.

## Cross-Experiment Summary

| Exp. | Method           | Key takeaway on BSDS boundary detection                                                                       |
|------|------------------|---------------------------------------------------------------------------------------------------------------|
| 1    | k-NN (scratch)   | Scaling fixes the metric; edge pixels are genuinely locally structured (best $k=5$ ).                         |
| 2    | Decision tree    | Rediscovered gradient thresholding; pruning $6322 \rightarrow 127$ leaves raises F1 $0.50 \rightarrow 0.62$ . |
| 3    | Naive Bayes      | Log transform repairs marginals; NB-vs-QDA measures the price of independence ( $\sim 0.007$ AUC).            |
| 4    | LogReg (scratch) | Matches scikit-learn; coefficients read as an edge-vs-texture filter.                                         |
| 5    | PCA / t-SNE      | 17 filters $\approx 8$ effective dimensions; classes overlap where humans disagree.                           |
| 6    | Outliers         | Extreme responses are signal, not noise; winsorisation changed AUC by exactly 0.                              |
| 7    | LR vs SVM        | SVM wins F1, LogReg wins AUC; errors are texture FPs and low-contrast FNs.                                    |